

Linux w Systemach Wbudowanych

3. Wykorzystanie linii GPIO jako wejść w systemach Raspberry Pi i Linux

Piotr ZAWADZKI

Wstęp

Cel opracowania

Celem tego opracowania jest przedstawienie metod wykorzystania linii GPIO jako wejść w systemie Linux na przykładzie Raspberry Pi. Omówimy narzędzia i API libgpiod oraz przestarzały interfejs sysfs, aby dostarczyć pełnej wiedzy na temat dostępnych rozwiązań i pomóc w wyborze najbardziej odpowiedniego podejścia w przyszłych projektach.

Krótkie wprowadzenie do GPIO

GPIO (ang. General Purpose Input/Output) to uniwersalne linie wejścia/wyjścia, które umożliwiają mikrokomputerom komunikację z urządzeniami zewnętrznymi. Na Raspberry Pi linie GPIO pozwalają na odczyt stanów logicznych z czujników, przycisków i innych urządzeń wejściowych, a także na sterowanie diodami LED, silnikami czy przełącznikami.

Podstawowe pojęcia i terminologia

Linia GPIO: pojedynczy pin na płycie Raspberry Pi, który może działać jako wejście lub wyjście cyfrowe. Wejście cyfrowe: konfiguracja linii GPIO pozwalająca na odczyt stanu logicznego (wysoki lub niski) urządzenia zewnętrznego. Wyjście cyfrowe: konfiguracja linii GPIO umożliwiająca ustawienie stanu logicznego na pinie w celu sterowania urządzeniem zewnętrznym. Pull-up/Pull-down resistor: rezystory podciągające stosowane w celu zapewnienia stabilnego stanu logicznego linii GPIO, gdy nie jest ona bezpośrednio sterowana. libgpiod: nowoczesne API i zestaw narzędzi użytkowych do obsługi linii GPIO w systemie Linux, zastępujące przestarzały interfejs sysfs. Interfejs sysfs: starszy sposób dostępu do linii GPIO poprzez system plików wirtualnych w Linuxie, obecnie uznawany za przestarzały.

Narzędzia i API libgpiod

Wprowadzenie do libgpiod

libgpiod to nowoczesna biblioteka oraz zestaw narzędzi wiersza poleceń służących do obsługi linii GPIO w systemach Linux. Została stworzona jako następca przestarzałego interfejsu sysfs i wykorzystuje nowy interfejs urządzeń znakowych /dev/gpiochip*. Dzięki libgpiod programiści mogą efektywnie zarządzać liniami GPIO, korzystając z udoskonalonych mechanizmów oferowanych przez jądro systemu Linux.

Instalacja libgpiod

Aby zainstalować libgpiod na Raspberry Pi z systemem Linux, można użyć menedżera pakietów apt:

```
sudo apt update
sudo apt install -y libgpiod-dev gpiod
```

Spowoduje to zainstalowanie zarówno bibliotek potrzebnych do programowania, jak i narzędzi wiersza poleceń.

Podstawowe komendy narzędzi libgpiod

libgpiod dostarcza zestaw narzędzi umożliwiających interakcję z liniami GPIO bez konieczności pisania kodu. Oto najważniejsze z nich:

gpiodetect

Wyświetla listę dostępnych układów GPIO w systemie.

```
gpiodetect
```

Przykładowy wynik:

```
gpiochip0 [pinctrl-bcm2835] (54 lines)
```

gpiointro

Podaje szczegółowe informacje o liniach GPIO dla wybranego układu.

```
gpioinfo gpiochip0
```

Można również wyświetlić informacje o wszystkich układach:

```
gpioinfo
```

gpioset

Ustawia stan wyjściowy na wybranych liniach GPIO.

```
gpioset gpiochip0 17=1
```

Powyższe polecenie ustawia linię GPIO17 na stan wysoki.

gpioget

Odczytuje stan linii GPIO skonfigurowanych jako wejścia.

```
gpioget gpiochip0 17
```

Zwraca aktualny stan logiczny linii GPIO17.

gpiofind

Znajduje numer linii GPIO na podstawie jej nazwy.

```
gpiofind "GPIO17"
```

Jeśli linia o podanej nazwie istnieje, zwróci jej numer.

gpiomon

Monitoruje zmiany stanu na wybranych liniach GPIO w czasie rzeczywistym.

```
gpiomon gpiochip0 21
```

Powyższe polecenie będzie na bieżąco wyświetlać zmiany stanu na linii GPIO17. Przykładowy wynik:

```
event: FALLING EDGE offset: 17 timestamp: [1234567.890]  
event: RISING EDGE offset: 17 timestamp: [1234568.123]
```

Można też monitorować wiele linii jednocześnie:

```
gpiomon gpiochip0 21 16
```

Polecenie jest szczególnie przydatne podczas debugowania i testowania połączeń GPIO.

Programowanie z użyciem libgpiod

Nowoczesna wersja libgpiod udostępnia API C++, które oferuje obiektowy interfejs dostępu do GPIO.

```
#include <gpiod.hpp>  
#include <iostream>  
#include <stdexcept>  
  
int main() {  
    try {  
        // Otwórz chip GPIO  
        gpiod::chip chip{"gpiochip0"};  
  
        // Uzyskaj linię GPIO  
        unsigned int line_num = 17;  
        auto line = chip.get_line(line_num);  
  
        // Skonfiguruj jako wejście  
        line.request({"gpio_input_example",  
                    gpiod::line_request::DIRECTION_INPUT,  
                    gpiod::line_request::FLAG_BIAS_PULL_UP});  
  
        // Odczytaj wartość  
        int value = line.get_value();  
        std::cout << "Stan linii GPIO" << line_num << ": " << value << std::endl;  
  
        // Zasoby zostaną automatycznie zwolnione dzięki RAII
```

```

    } catch (const std::exception& e) {
        std::cerr << "Błąd: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}

```

Aby skompilować powyższy kod:

```
g++ -o gpio_input_example gpio_input_example.cpp -lgpiodcxx
```

Główne zalety API C++:

- Obsługa wyjątków zamiast kodów błędów.
- Automatyczne zarządzanie zasobami (RAII).
- Bardziej zwięzły i bezpieczniejszy kod.
- Wsparcie dla nowoczesnych funkcji C++.

Let me outline the documentation for the deprecated sysfs GPIO interface.

Przestarzały interfejs sysfs

Wprowadzenie do sysfs

Interfejs sysfs dla GPIO, dostępny w `/sys/class/gpio/`, był standardowym sposobem kontroli GPIO w Linuxie przed wprowadzeniem libgpiod. Jest obecnie oznaczony jako przestarzały i zostanie usunięty w przyszłych wersjach jądra.

Konfiguracja GPIO jako wejścia za pomocą sysfs

Podstawowa ścieżka dostępu do GPIO przez sysfs:

```

/sys/class/gpio/
├── export
├── unexport
├── gpioN/
│   ├── direction
│   ├── value
│   └── edge

```

Eksportowanie linii GPIO

Przed użyciem GPIO należy je “wyeksportować”:

```

# Eksportowanie GPIO17
echo "17" > /sys/class/gpio/export

```

Ustawianie kierunku linii

Po wyeksportowaniu, ustawiamy kierunek na wejście:

```
echo "in" > /sys/class/gpio/gpio17/direction
```

Odczyt stanu linii GPIO

Stan linii można odczytać z pliku `value`:

```
cat /sys/class/gpio/gpio17/value
```

Przykładowy kod w Bash

```

#!/bin/bash

GPIO=17

# Eksportuj GPIO jeśli nie jest już wyeksportowane
if [ ! -d "/sys/class/gpio/gpio$GPIO" ]; then
    echo "$GPIO" > /sys/class/gpio/export
    sleep 0.1
fi

# Ustaw jako wejście

```

```
echo "in" > /sys/class/gpio/gpio$GPIO/direction
```

```
# Odczytaj wartość
value=$(cat /sys/class/gpio/gpio$GPIO/value)
echo "Stan GPIO$GPIO: $value"
```

```
# Zwolnij GPIO
echo "$GPIO" > /sys/class/gpio/unexport
```

Przykładowy kod w C++

```
#include <iostream>
#include <fstream>
#include <string>
#include <stdexcept>

class GPIOInput {
private:
    int pin;

public:
    GPIOInput(int gpio_pin) : pin(gpio_pin) {
        export_gpio();
        set_direction();
    }

    ~GPIOInput() {
        unexport_gpio();
    }

    void export_gpio() {
        std::ofstream export_file("/sys/class/gpio/export");
        if (!export_file) throw std::runtime_error("Nie można otworzyć pliku export");
        export_file << pin;
    }

    void set_direction() {
        std::string path = "/sys/class/gpio/gpio" + std::to_string(pin) + "/direction";
        std::ofstream direction_file(path);
        if (!direction_file) throw std::runtime_error("Nie można ustawić kierunku");
        direction_file << "in";
    }

    int read_value() {
        std::string path = "/sys/class/gpio/gpio" + std::to_string(pin) + "/value";
        std::ifstream value_file(path);
        if (!value_file) throw std::runtime_error("Nie można odczytać wartości");
        char value;
        value_file >> value;
        return value - '0';
    }

    void unexport_gpio() {
        std::ofstream unexport_file("/sys/class/gpio/unexport");
        if (!unexport_file) return; // Ignoruj błąd przy zwalnianiu
        unexport_file << pin;
    }
};

int main() {
    try {
        GPIOInput gpio(17);
        std::cout << "Stan GPIO17: " << gpio.read_value() << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Błąd: " << e.what() << std::endl;
        return 1;
    }
    return 0;
}
```

Let me outline a comparison between libgpiod and sysfs GPIO interfaces.

Porównanie libgpiod i sysfs

Zalety i wady obu podejść

libgpiod

Zalety:

- Atomowe operacje na liniach GPIO
- Lepsza obsługa współbieżności
- Nowoczesne API w C i C++
- Narzędzia wiersza poleceń
- Możliwość rezerwacji wielu linii w jednej operacji
- Obsługa nazw linii GPIO
- Lepsza wydajność

Wady:

- Wymaga nowszych wersji jądra Linux (≥ 4.8)
- Mniej dokumentacji i przykładów
- Konieczność linkowania z biblioteką

sysfs

Zalety:

- Prosta implementacja
- Łatwy dostęp z poziomu skryptów
- Działa na starszych systemach
- Dobra dokumentacja i wiele przykładów

Wady:

- Przestarzały interfejs
- Brak atomowości operacji
- Problemy ze współbieżnością
- Niższa wydajność
- Ograniczona funkcjonalność
- Oznaczony jako deprecated

Przyszłość obsługi GPIO w systemie Linux

- Interfejs sysfs zostanie całkowicie usunięty w przyszłych wersjach jądra
- libgpod jest oficjalnym następcą i zalecanym interfejsem
- Rozwój libgpod koncentruje się na:
 - Rozszerzeniu funkcjonalności
 - Poprawie wydajności
 - Lepszej integracji z systemem
 - Wsparciu dla nowych funkcji sprzętowych
- Zalecana migracja do libgpod dla nowych projektów

Przykład różnicy w kodzie:

```
// libgpod - nowoczesne podejście
gpod::chip chip("gpiochip0");
auto line = chip.get_line(17);
line.request({"app", gpod::line_request::DIRECTION_INPUT});
int value = line.get_value();
```

```
// sysfs - stare podejście
std::ofstream export_file("/sys/class/gpio/export");
export_file << "17";
std::ofstream direction_file("/sys/class/gpio/gpio17/direction");
direction_file << "in";
std::ifstream value_file("/sys/class/gpio/gpio17/value");
char value;
value_file >> value;
```

Przepraszam za błąd. Poprawię konfigurację linii GPIO, aby odpowiednio obsługiwały wyzwalanie zboczem.

Konfiguracja przycisku chwilowego jako wejścia za pomocą dedykowanego sterownika gpio_keys

Koncepcja działania sterownika gpio_keys

Sterownik gpio_keys jest częścią jądra Linux, który umożliwia obsługę przycisków podłączonych do linii GPIO jako standardowych urządzeń wejściowych. Sterownik ten mapuje zdarzenia fizyczne (np. wciśnięcie przycisku) na zdarzenia

wejściowe systemu Linux, takie jak naciśnięcie klawisza na klawiaturze. Dzięki temu przyciski GPIO mogą być obsługiwane przez standardowe mechanizmy wejściowe systemu, co upraszcza ich integrację z aplikacjami użytkownika.

Zasada działania

1. Definicja przycisku w Device Tree:

- Konfiguracja przycisku odbywa się poprzez plik Device Tree (DT), który opisuje sprzęt systemu.
- W pliku DT definiujemy węzeł `gpio-keys`, który zawiera informacje o przyciskach, takie jak numer GPIO, typ zdarzenia i kod klawisza.

2. Obsługa zdarzeń przez jądro:

- Sterownik `gpio_keys` monitoruje linie GPIO skonfigurowane w DT.
- Gdy wykryje zmianę stanu (np. wciśnięcie lub zwolnienie przycisku), generuje odpowiednie zdarzenie wejściowe.
- Zdarzenia te są przekazywane do systemu wejściowego Linux, co pozwala na ich obsługę przez aplikacje użytkownika.

3. Integracja z systemem wejściowym:

- Zdarzenia generowane przez `gpio_keys` są widoczne jako standardowe zdarzenia wejściowe (np. klawisze klawiatury).
- Aplikacje mogą korzystać z tych zdarzeń bez konieczności bezpośredniego dostępu do GPIO.

Przykład konfiguracji w Device Tree

Poniżej znajduje się przykładowa konfiguracja przycisku chwilowego w pliku Device Tree:

```
{
    gpio-keys {
        compatible = "gpio-keys";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_gpio_keys>;

        button@0 {
            label = "gpio_button1";
            gpios = <&gpio 17 GPIO_ACTIVE_HIGH>;
            linux,code = <KEY_ENTER>;
            debounce-interval = <10>;
        };

        button@1 {
            label = "gpio_button2";
            gpios = <&gpio 18 GPIO_ACTIVE_HIGH>;
            linux,code = <KEY_ESC>;
            debounce-interval = <10>;
        };
    };
};

&gpio {
    pinctrl_gpio_keys: gpio_keys {
        pinmux = <PINMUX_GPIO_17>, <PINMUX_GPIO_18>;
        bias-pull-down;
    };
};
```

Wyjaśnienie konfiguracji

• Węzeł `gpio-keys`:

- `compatible = "gpio-keys";`: Określa, że węzeł jest obsługiwany przez sterownik `gpio_keys`.
- `pinctrl-names` i `pinctrl-0`: Definiują konfigurację pinów GPIO.

• Węzeł `button@0` i `button@1`:

- `label = "gpio_button1";`: Etykieta pierwszego przycisku.
- `gpios = <&gpio 17 GPIO_ACTIVE_HIGH>;`: Określa numer GPIO (17) i aktywny stan wysoki dla pierwszego przycisku.
- `linux,code = <KEY_ENTER>;`: Kod klawisza generowanego przez pierwszy przycisk (np. `KEY_ENTER`).
- `debounce-interval = <10>;`: Czas debouncingu w milisekundach.
- Analogicznie dla drugiego przycisku (`gpio_button2`, GPIO 18, `KEY_ESC`).

• Węzeł `&gpio`:

- `pinctrl_gpio_keys`: Definiuje konfigurację pinów GPIO.
- `pinmux = <PINMUX_GPIO_17>, <PINMUX_GPIO_18>;`: Określa, że piny 17 i 18 są używane jako GPIO.
- `bias-pull-down`: Ustawia rezystor pull-down na pinach GPIO.

Konfiguracja w `/boot/firmware/config.txt`

Aby załadować powyższą konfigurację Device Tree, należy dodać odpowiedni wpis w pliku `/boot/firmware/config.txt`. Poniżej znajduje się przykład konfiguracji dla dwóch przycisków:

```
dtoverlay=gpio-key,gpio=17,active_low=0,gpio_pull=down,label=button1,keycode=28,debounce=10
dtoverlay=gpio-key,gpio=18,active_low=0,gpio_pull=down,label=button2,keycode=1,debounce=10
```

Wyjaśnienie konfiguracji w `config.txt`

- `dtoverlay=gpio-key`: Określa użycie nakładki Device Tree `gpio-key`.
- `gpio=17`: Numer GPIO dla pierwszego przycisku.
- `active_low=0`: Określa, że aktywny stan jest wysoki.
- `gpio_pull=down`: Ustawia rezystor pull-down.
- `label=button1`: Etykieta przycisku.
- `keycode=28`: Kod klawisza generowanego przez pierwszy przycisk (np. `KEY_ENTER`).
- `debounce=10`: Czas debouncingu w milisekundach.
- Analogicznie dla drugiego przycisku (`gpio=18`, `label=button2`, `keycode=1` (np. `KEY_ESC`)).

Podsumowanie

Sterownik `gpio_keys` upraszcza obsługę przycisków podłączonych do GPIO, mapując zdarzenia fizyczne na standardowe zdarzenia wejściowe systemu Linux. Konfiguracja odbywa się poprzez plik Device Tree oraz plik `/boot/firmware/config.txt`, co pozwala na łatwą integrację z systemem i aplikacjami użytkownika.

Popularne urządzenia wejścia

Przycisk chwilowy (push button)

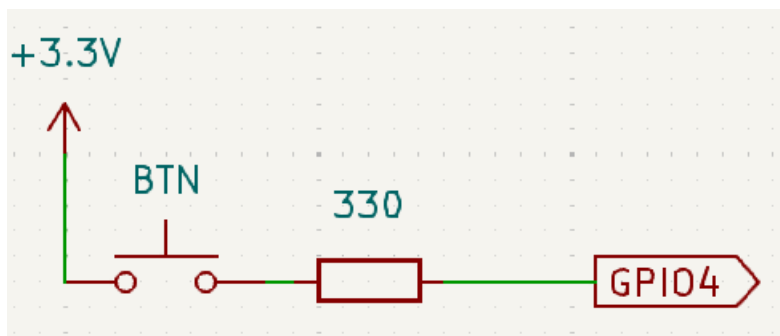
Opis funkcjonalny

Przycisk chwilowy to prosty mechaniczny przełącznik, który łączy dwa styki, gdy jest wciśnięty, i rozłącza je, gdy jest zwolniony. Jest często używany do generowania sygnałów wejściowych w systemach elektronicznych, takich jak uruchamianie funkcji, resetowanie urządzeń czy wprowadzanie danych przez użytkownika.

Zasada działania

- W stanie spoczynkowym przycisk jest rozarty, a GPIO jest połączone z masą (GND) przez rezystor pull-down, co oznacza, że linia GPIO jest w stanie niskim (0).
- Wciśnięcie przycisku powoduje zwarcie styków, co łączy GPIO z napięciem 3.3V, powodując, że linia GPIO przechodzi w stan wysoki (1).
- Prąd płynie tylko wtedy, gdy przycisk jest wciśnięty, co minimalizuje zużycie energii.

Schemat podłączenia:



push button circuit

Przykładowy kod w C++ z wyzwalaniem zboczem

```
#include <gpiod.hpp>
```

```
gpiod::chip chip("gpiochip0");
gpiod::line line = chip.get_line(lineNum);
```

```

line.request({"button",
    gpod::line_request::EVENT_BOTH_EDGES
    gpod::line_request::FLAG_BIAS_PULL_DOWN});
auto event = line.event_wait(std::chrono::milliseconds(100));
if (event) {
    auto event = line.event_read();
    if (event.event_type == gpod::line_event::RISING_EDGE) {
        // button pressed, see the hardware configuration
    } else {
        // button released
    }
}
// not required because of RAI
// line.release()

```

Gdy przycisk jest skonfigurowany jako gpio-key

```
dtoverlay=gpio-key,gpio=4,active_low=0,gpio_pull=down,label=lsw-key,keycode=16,debounce=200
```

zdarzenia przezeń generowane można odbierać z systemowej kolejki zdarzeń. Aplikacja nie musi wtedy posiadać praw do odczytu stanu linii GPIO. Mówiąc obrazowo, przycisk staje się klawiszem na klawiaturze i tak też należy go w programie traktować.

```

const char* device = "/dev/input/event0";
int fd = open(device, O_RDONLY);
int epoll_fd = epoll_create1(0);

struct epoll_event event;
event.events = EPOLLIN;
event.data.fd = fd;

epoll_ctl(epoll_fd, EPOLL_CTL_ADD, fd, &event)

while (true) {
    struct epoll_event ev;
    int nfds = epoll_wait(epoll_fd, &ev, 1, -1);

    if (nfds > 0 && (ev.events & EPOLLIN)) {
        struct input_event input_ev;
        read(ev.data.fd, &input_ev, sizeof(input_ev));
        std::cout << "Event: time " << input_ev.time.tv_sec << "." << input_ev.time.tv_usec
            << ", type " << input_ev.type
            << ", code " << input_ev.code
            << ", value " << input_ev.value << std::endl;
    }
}

close(fd);
close(epoll_fd);

```

Enkoder obrotowy (rotary encoder)

Opis funkcjonalny

Enkoder obrotowy to urządzenie, które przekształca ruch obrotowy na sygnały elektryczne. Jest używany do precyzyjnego określania pozycji kątowej, kierunku obrotu oraz prędkości obrotowej. Enkodery obrotowe są powszechnie stosowane w interfejsach użytkownika, takich jak pokrętła regulacyjne, oraz w systemach sterowania ruchem.

Zasada działania

- Enkoder generuje dwie sekwencje impulsów (CLK/SIA i DT/SIB), które są przesunięte w fazie.
- w zależności od wykrytego zbocza na linii CLK i stanu linii DT tabela dekodowania obrotu jest następująca

ROT CCW CCW CW CW

	CLK rising	CLK falling	rising	falling
DT	0	1	1	0

Niektóre enkodery utrzymują linię CLK w stabilnym położeniu, co oznacza, że zdarzenia rising and falling występują zawsze w parze. Należy wtedy dekodować co drugie zdarzenie, tzn. pominąć kolumnę 3 i 5 tabeli dekodowania.

Sekwencje zdarzeń

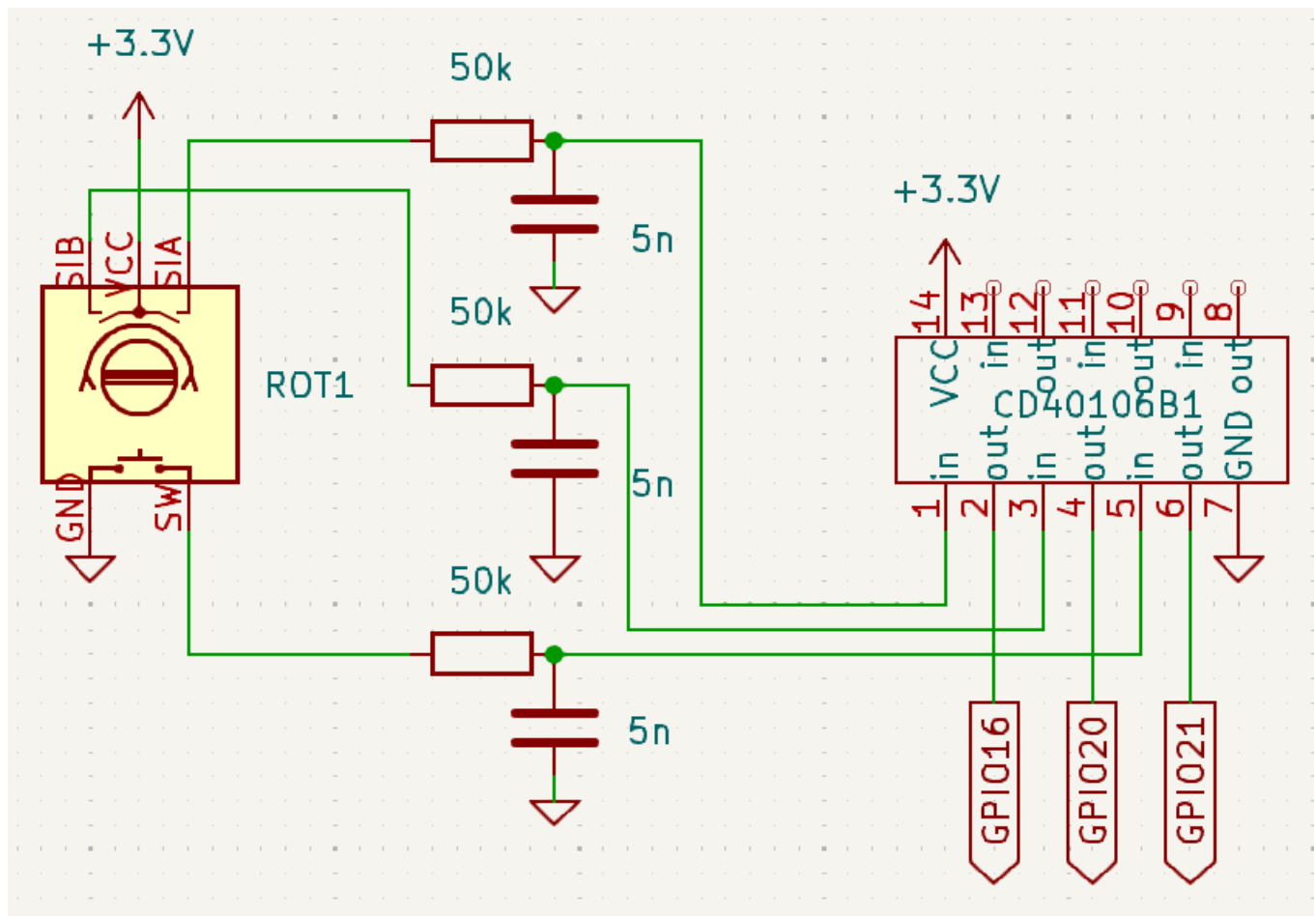
- Obrót w prawo (clockwise/CW):
 1. CLK przechodzi z 0 do 1, gdy DT jest 1.

2. DT przechodzi z 1 do 0.
3. CLK przechodzi z 1 do 0, gdy DT jest 0.
4. DT przechodzi z 0 do 1.

• Obrót w lewo (odwrotna sekwencja):

1. CLK przechodzi z 0 do 1, gdy DT jest 0.
2. DT przechodzi z 0 do 1.
3. CLK przechodzi z 1 do 0, gdy DT jest 1.
4. DT przechodzi z 1 do 0.

Schemat podłączenia:



rotary encoder

Proszę zauważyć, że przedstawione przebiegi i sekwencje dotyczą sytuacji gdy:

- Enkoder znajduje się zasilanej płytce zawierającej interfejs elektryczny, w stanie neutralnym wszystkie sygnały są w stanie wysokim.
- W celu eliminacji "dzwonienia styków" zastosowano filtr dolnoprzepustowy i odwracający przerzutnik Schmitta, zatem na wejściach GPIO w stanie neutralnym mamy stan niski.

Przykładowy kod w C++ z wyzwalaniem zboczem

```
#include <gpod.hpp>
```

```
gpod::chip Schip("gpiochip0");
gpod::line SIA = chip.get_line(16);
SIA.request(
    "rotary_SIA",
    gpod::line_request::EVENT_BOTH_EDGES,
    gpod::line_request::FLAG_BIAS_PULL_DOWN
);
gpod::line SIB = chip.get_line(20);
SIB.request(
    "rotary_SIB",
    gpod::line_request::DIRECTION_INPUT,
    gpod::line_request::FLAG_BIAS_PULL_DOWN
);
```

```
auto event = SIA.event_wait(std::chrono::milliseconds(10));
```

```

if (event) {
    auto SIA_event = SIA_line.event_read();
    int SIB_Value = SIB_line.get_value();
    if (SIA_event.event_type == gpio::line_event::RISING_EDGE && SIB_Value == 1) {
        // Clockwise rotation
    } else if (SIA_event.event_type == gpio::line_event::FALLING_EDGE && SIB_Value == 1) { // CounterClockwise rotation
        // decrease the proximity threshold
        auto newThreshold = std::max(appState.proximityThreshold.load() - proximityThresholdDelta, proximityThresholdMin);
        appState.proximityThreshold.store(newThreshold);
    }
}
}

```

Dalmierz akustyczny HC-SR04

Opis funkcjonalny

Dalmierz akustyczny HC-SR04 to czujnik odległości, który wykorzystuje ultradźwięki do pomiaru dystansu do obiektu. Jest powszechnie stosowany w robotyce, systemach parkowania oraz w aplikacjach wymagających pomiaru odległości bezkontaktowego.

Zasada działania

1. Sygnał TRIGGER:

- Stan początkowy: obie linie TRIGGER i ECHO w stanie niskim.
- Zbocze narastające (impuls o szerokości co najmniej 10µs) na linii TRIGGER inicjuje pomiar.
- Moduł wysyła 8 impulsów ultradźwiękowych o częstotliwości 40kHz.
- Linia ECHO przechodzi w stan wysoki.

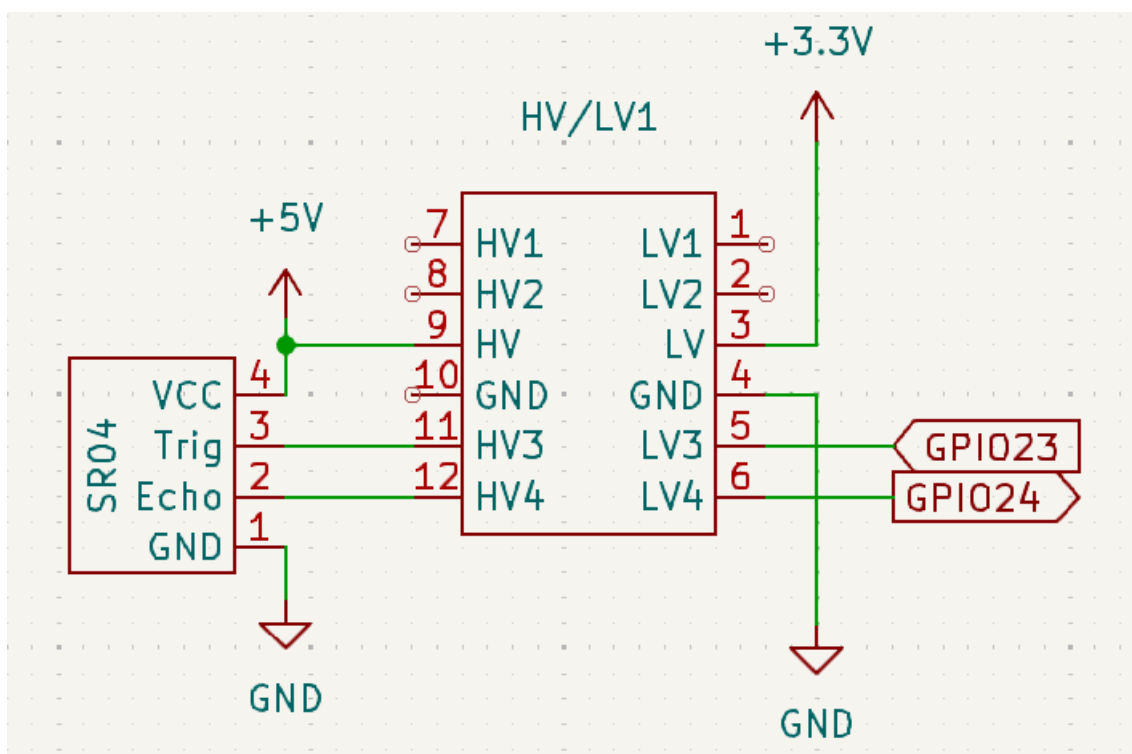
2. Sygnał ECHO:

- Pozostaje wysoka do momentu odebrania echa.
- Po odebraniu echa I
- Czas trwania impulsu ECHO jest proporcjonalny do odległości.
- Przeliczenie: $\text{odległość} = (\text{czas_ECHO} \times \text{prędkość_dźwięku}) / 2$.

3. Interfejs elektryczny:

- Zasilanie +5V.
- TRIGGER: wejście 3.3V/5V.
- ECHO: wyjście 5V (wymaga dzielnika napięcia dla GPIO 3.3V).
- Dzielnik R1=470Ω, R2=1kΩ zapewnia konwersję poziomów.

Schemat podłączenia:



HC-SR04 circuit

Przykładowy kod w C++ z wyzwalaniem zboczem

```
#include <gpod.hpp>

gpod::chip chip("gpiochip0");
trigger_line = chip.get_line(23);
trigger_line.request(
    "SR04-Trigger"
    ,gpod::line_request::DIRECTION_OUTPUT
    ,0
);
trigger_line.set_value(0);
echo_line_ = chip.get_line(24);
echo_line.request(
    "SR04-Echo"
    ,gpod::line_request::EVENT_BOTH_EDGES
    ,gpod::line_request::FLAG_BIAS_PULL_DOWN
);
trigger_line.set_value(1);
usleep(20);
trigger_line.set_value(0);

auto rising_time = std::chrono::steady_clock::time_point::min();
auto falling_time = std::chrono::steady_clock::time_point::min();

auto event = echo_line.event_wait(5);
if (event) {
    auto evt = echo_line.event_read();
    if (evt.event_type == gpod::line_event::RISING_EDGE) {
        // Check if the line value is high
        if (echo_line.get_value() == 1) {
            rising_time = std::chrono::steady_clock::now();
        }
    }
}
auto event = echo_line.event_wait(50);
if (event) {
    auto evt = echo_line.event_read();
    if (evt.event_type == gpod::line_event::FALLING_EDGE) {
        // Check if the line value is low
        if (echo_line.get_value() == 0) {
            falling_time = std::chrono::steady_clock::now();
        }
    }
}

int duration_us = std::chrono::duration_cast<std::chrono::microseconds>(falling_time - rising_time).count();
double duration_seconds = (1.0*duration_us) / 1'000'000.0;
double speed_of_sound = 343.0; // m/s
auto distance_m = (duration_seconds * speed_of_sound) / 2.0; // round trip
```